

Design and Synthesis of D Flip Flop using Yosys

INTRODUCTION:

In this lab, the focus is on synthesizing a D Flip-Flop (D_FF) using Yosys, an open-source RTL synthesis tool. The D Flip-Flop serves as a fundamental sequential building block in digital design, used to store a single bit of data synchronously on clock edges while enabling reliable state retention and timing control. Yosys reads the Verilog description of the D_FF, elaborates the design, infers the required latches, clock gating, and reset logic, and maps them to standard cells for efficient implementation in terms of area, speed, and power.

APPLICATIONS:

A D Flip-Flop is used in digital systems for data synchronization and storage, typically for clock domain crossing, pipeline registers, and state machines. It is commonly applied in processors, memory interfaces, and control logic where precise timing alignment and glitch-free data transfer are required.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

1.1: Create a directory for your lab: `mkdir <project_name>`

1.2: Open lab directory: `cd <Project_name>`

1.3: Create directory for project: `mkdir d_ff`

1.4: Open project directory: `cd d_ff`

Step 2: Create Design file

2.1: Open gvim editor for design with command: `gvim d_ff.v`

RTL Design Code:

```
module d_ff (  
    input clk,  
    input rst,  
    input d,
```

```
output reg q
);
always @(posedge clk) begin
    if(rst)
        q <= 0;
    else
        q <= d;
end
endmodule
```

2.2: Save the file using command- :wq!

Step 3: Synthesize the Design with Yosys

Invoke Yosys in a new terminal by the following command: `yosys`

3.1: Use the following command to Synthesize:

```
# Read the Verilog source file
read_verilog d_ff.v

# Set the top-level module and check hierarchy
hierarchy -check -top d_ff

# high level stuff
proc; opt; fsm; opt; memory; opt

# mapping to internal cell library
techmap; opt

# mapping flip-flops to mycells.lib
dfflibmap -liberty osu018_stdcells.lib

# mapping logic to mycells.lib
abc -liberty osu018_stdcells.lib
```

Clean up unused cells and signals after mapping

clean

Write the final synthesized gate-level netlist to a new Verilog file

write_verilog d_ff_synth.v

Step 4: View Output File (Netlist)

gvim d_ff_synth.v

```
13. Generating Graphviz representation of design.
Writing dot description to '/home/coe-2/.yosys_show.dot'.
Dumping module d_ff to page 1.
Exec: { test -f '/home/coe-2/.yosys_show.dot.pid' && fuser -s '/home/coe-2/.yosys_show.dot.pid'; } || ( echo $$ >&3; exec xdot '/home/coe-2/.yosys_show.dot'; ) 3> '/home/coe-2/.yosys_show.dot.pid' &

14. Executing Verilog backend.
Dumping module '\d_ff'.

15. Printing statistics.

=== d_ff ===
Number of wires:          6
Number of wire bits:      6
Number of public wires:   4
Number of public wire bits: 4
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          3
  DFFPOSX1                 1
  INVX1                    1
  NOR2X1                   1

Chip area for module '\d_ff': 136.000000
```

OBSERVATION:

A D Flip-Flop (D_FF) is a fundamental sequential circuit element that captures and holds a single bit of data (D input) on the active edge of a clock signal, transferring it to the output (Q) while ignoring subsequent input changes until the next clock edge. It provides stable state storage with optional asynchronous or synchronous reset capabilities, ensuring predictable timing behaviour critical for sequential logic design. This makes it an essential building block in registers, counters, shift registers, and pipeline stages, where reliable data synchronization across clock domains and glitch-free operation are paramount.

OUTPUT:

Synthesized Netlist (d_ff_synth.v)

CONCLUSION:

The Yosys synthesis output confirms the correct implementation of the D Flip-Flop, showing the RTL design was successfully mapped to a gate-level netlist. This netlist preserves the

intended sequential structure, with distinct latch and clock gating logic for state storage, all governed by a properly synthesized reset and timing control circuitry.

EchipHub.in